

PicoPata

Ropa infantil

Del prototipo al código

Laura Rubio Gallardo
2ºDAM

MEDAC
Instituto Oficial de Formación Profesional

Índice

1. Análisis del diseño 3
1.1. Árbol de Figma 3
1.2. Árbol de Android Studio 4
2. Ficha de especificaciones 6
2.1. Colores 6
2.2. Tipografías 6
2.3. Medidas claves 7
3. Assets 8
3.1. Inventario 8
3.2. Capturas de pantalla de Android Studio 9
4. Reflexión 12
4.1. ¿Qué decisión del análisis o de la preparación en Figma te ha resultado más difícil? ¿Por qué? 12
4.2. Has pasado tus colores y fuentes a recursos. ¿Qué tendrías que cambiar si mañana el cliente modifica el color principal? ¿Y si no hubieras usado recursos? 12
4.3. ¿Cuánto ocupaba tu imagen en PNG y cuánto en WebP? ¿Qué conclusión sacas? 13
4.4. Conclusión 13

MEDAC

Del prototipo al código

Enlace Figma

1. Análisis del diseño

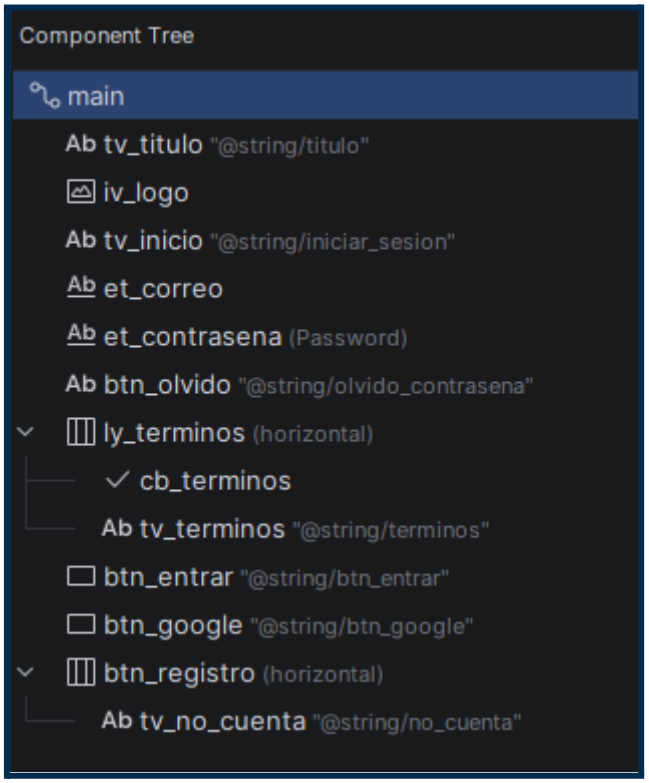
1.1. Árbol de Figma

Descomponemos la pantalla en un árbol de bloques y lo comparamos con el panel de capas.



Árbol del Figma

1.2. Árbol de Android Studio



Árbol de Android Studio

Qué no se programa:

- **La interfaz nativa del sistema:** Esto es la barra de estado superior, lo que contiene, el reloj, el nivel de batería, los iconos de notificaciones. Y la barra de navegación inferior, donde están los gestos o botones virtuales. Esto no forma parte del árbol de nuestra aplicación. Son componentes superpuestos y los gestiona el sistema operativo Android. Además en nuestro archivo XML, en el activity_main.xml, consiste en respetar los márgenes seguros, Safe Areas, para que nuestro diseño no quede oculto debajo de estos elementos del sistema.
- **Los recursos gráficos estáticos (Assets):** Estos son los recursos gráficos estáticos como el logo principal (img_logo), y el icono de Google (ic_google). Se exportan y se cargan directamente desde la carpeta res/drawable/ son dibujarlos por código. Se exportan en los formatos óptimos de compresión (WebP), o escalabilidad matemática (VectorDrawable).

MEDAC

Qué se superpone:

- **Composición interna de los botones de icono:** Esto es el icono vectorial de Google se superpone y convive junto al texto del área táctil del botón btn_google. El gráfico vectorial (ic_google) no se crea como un ImageView independiente, sino que está integrado en el mismo nodo XML junto a la etiqueta de texto.
- **Capas dinámicas de estado en los campos de textos:** Con esto nos referimos a el texto que escribe el usuario, se superpone al fondo de la caja de texto en et_correo y et_contrasena, ocultando así el texto de sugerencia original (hint).

Qué se repite:

- **La estructura geométrica y la tipografía:** Los campos de formulario, et_correo y et_contrasena, comparten la misma estructura geométrica, mismo ancho y mismo alto (328 dp x 56 dp) y el estilo tipográfico de estos. Este estilo lo podemos encontrar en el archivo style.xml, aplicando un estilo unificado para no duplicar líneas de código, evitando el hardcoding.
- **Patrón de interacción en botones principales:** Los botones principales, como btn_entrar y btn_google, repiten el mismo patrón estructural de componente, el ancho y alto (328 dp x 48 dp). Su estructura se repite para mantener la consistencia en la experiencia de usuario y asegura que las zonas táctiles sean idénticas y predecibles.
- **Agrupaciones lógicas en ejes secundarios:** Los contenedores horizontales como ly_terminos y btn_registro, repiten la estructura de LinearLayout para alinear elementos secundarios en fila. Utilizan contenedores secundarios dentro del Constraint Layout principal para alinear los elementos en fila, repitiendo este patrón estructural cada vez que se necesita tratar varios subelementos.

2. Ficha de especificaciones

2.1. Colores

Nombre (función)	Hex	Dónde se usa
Surface (Fondo general)	#F6FAFE	fondos de pantallas
Primary	#1C6586	fondo del botón principal btn_entrar
On Surface (Texto principal)	#181C1F	Titulos, “PicoPato”, “Inicio sesión” tv_inicio, textos principales, también en “Continuar con Google”
On Primary (Texto sobre el primario)	#FFFFFF	Textos para el interior del botón de entrar btn
Tertiary Container (Fondo de google)	#E6DEFF	Fondo del botón secundario btn_google
On Surface Variant (Texto secundario)	#41484D	Placeholders de et_correo y et_contrasena
Outline (Borde / Contorno)	#C6C6C6	Borde del btn_google y cajas de texto

2.2. Tipografías

Elemento (ID)	Fuente	Peso	Tamaño (sp)	Interlineado (sp)	Color
tv_titulo	Roboto	400	22	32	On Surface
tv_inicio	Roboto	400	22	32	On Surface
et_correo	Roboto	400	16	24	On Surface Variant
et_contrasena	Roboto	400	16	24	On Surface

MEDAC

					Variant
btn_entrar	Roboto	500	14	20	On Primary
btn_google	Roboto	500	14	20	On Surface
tv_terminos	Roboto	400	14	20	On Surface Variant
btn_olvido	Roboto	400	14	20	On Surface Variant
btn_registro	Roboto	400	14	20	On Surface Variant

2.3. Medidas claves

Elemento	Ancho x Alto	Radio	Padding/margen	Separación (gap)
tv_titulo	328 x 28	0	centrado, 16dp con márgenes laterales, 59 márgen superior	0
iv_logo	215 x 217	32	centrado, 115dp con margen superior, 73dep márgenes laterales	0
tv_inicio	360 x 40	0	centrado, 364dp margen superior	0
et_correo	328 x 56	8	centrado, 400dp márgen superior, 16dp margen lateral	0
et_contrasena	328 x 56	8	centrado, 472dp margen superior, 16dp margen	0

MEDAC

			lateral	
btn_olvido	328 x 48	0	centardo, 528dp margen superior, 16dp margen lateral	0
ly_terminos	328 x 48	0	centrado, 568dp margen superior, 16 margen lateral	0
btn_entrar	328 x 48	24	centrado, 616dp margen superior, 16 margen lateral	0
btn_google	328 x 48	24	centrado, 680dp margen superior, 16 margen lateral	12
btn_registro	328 x 48	0	centrado, 728dp margen superior, 16 margen lateral	0

3. Assets

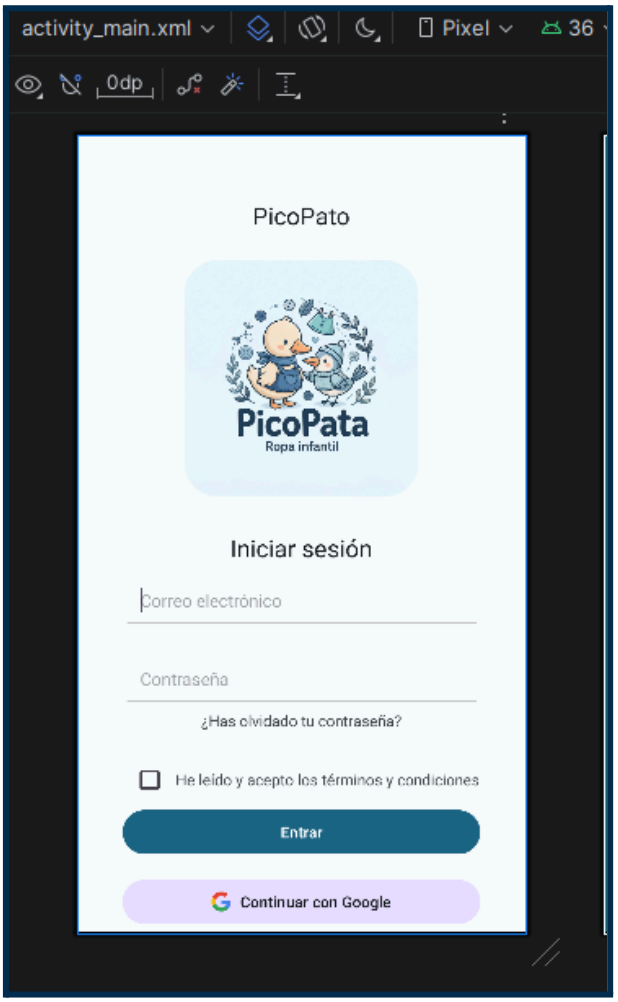
3.1. Inventario

Capa en Figma	¿Se exporta?	Formato	Nombre en Android	Carpera
Logo de la marca	Si	PNG — WebP, para optimizar el peso de instalador y consumo de memoria	img_logo	res/drawable-xxhdpi/
Icono de Google	Si, pero desde una fuente oficial, no desde figma	SVG — Vector Drawable, código matemático para garantizar nitidez	ic_google	res/drawable/

MEDAC

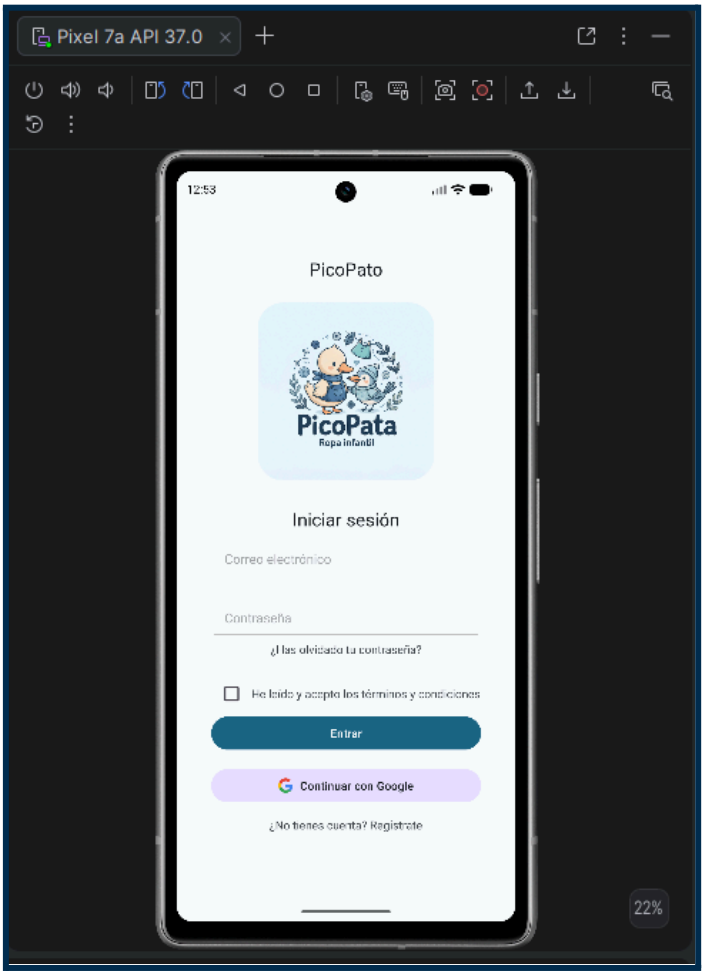
Fondo botón entrar	No, hay que programarlo	Código XML, shape	bg_btn_entrar	res/drawable/
Fondo botón google	No, hay que programarlo	Código XML, shape	bg_btn_google	res/drawable/
Fondo botón correo	No, hay que programarlo	Código XML, shape	bg_input_correo	res/drawable/
Fondo input contraseña	No, hay que programarlo	Código XML, shape	bg_input_contrasena	res/drawable/

3.2. Capturas de pantalla de Android Studio

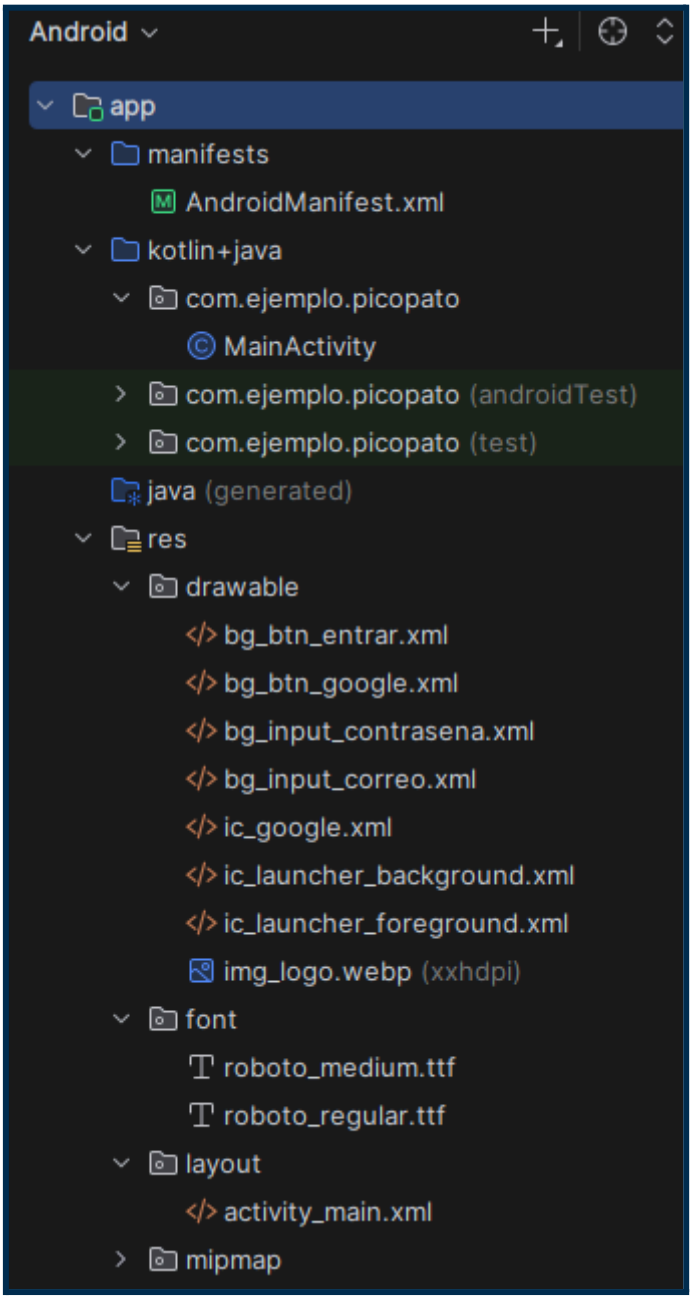


activity_main.xml

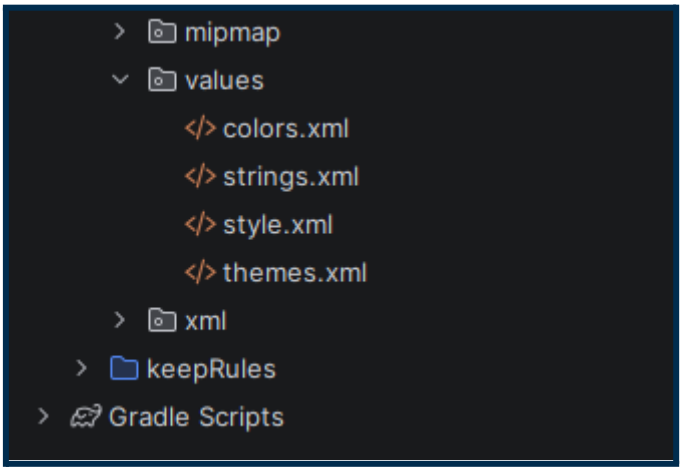
MEDAC



emulador de la app


Android
app
manifests
AndroidManifest.xml
kotlin+java
com.ejemplo.picopato
MainActivity
com.ejemplo.picopato (androidTest)
com.ejemplo.picopato (test)
java (generated)
res
drawable
bg_btn_entrar.xml
bg_btn_google.xml
bg_input_contrasena.xml
bg_input_correo.xml
ic_google.xml
ic_launcher_background.xml
ic_launcher_foreground.xml
img_logo.webp (xxhdpi)
font
roboto_medium.ttf
roboto_regular.ttf
layout
activity_main.xml
mipmap

MEDAC



jerarquía de Android Studio

4. Reflexión

4.1. ¿Qué decisión del análisis o de la preparación en Figma te ha resultado más difícil? ¿Por qué?

Lo más difícil de la actividad ha sido analizar toda la jerarquía necesaria para hacer la pantalla y analizar y ver cuales de todos los elementos tenía que exportar como un gráfico y cuales para programar el código, dando por hecho que no puede haber nada hardcodeado. Con el icono de Google también tuve muchos problemas al principio, no pude exportarlo directamente desde Figma, tuve que buscarlo en internet, igualmente no pude exportarlo y por último busqué el gráfico vectorial (SVG) oficial de la marca para importarlo como VectorDrawable y así garantizar la calidad de la imagen

4.2. Has pasado tus colores y fuentes a recursos. ¿Qué tendrías que cambiar si mañana el cliente modifica el color principal? ¿Y si no hubieras usado recursos?

Lo bueno de que no haya hardcodeo y que esté todo bien centralizado, en eset caso en colors.xml, es que solo tendría que cambiar una línea de código en el archivo de res/values/colors.xml, modificando el color que quieta en código hexadecimal, y solo con

eso se aplicará de forma instantánea y automáticamente en toda la aplicación. En caso de no haber usado los recursos, tendría que ir archivo por archivo y línea por línea buscando dónde he escrito el color a mano para sustituirlo, perdiendo mucho tiempo y riesgo de olvidar algún elemento.

4.3. ¿Cuánto ocupaba tu imagen en PNG y cuánto en WebP? ¿Qué conclusión sacas?

La imagen original de los patos en PNG ocupa 579 KB. Cuando la convertimos a WebP desde Android Studio, el tamaño se reduce a 28.9 KB, quedándose en tan solo un 4% de su peso original. El pasarlo a formato WebP hace que que se reduzca un montón sin perder la calidad visual de la imagen, cosa que es esencial para que el archivo de instalación peso menos y reducir el uso de la memoria RAM.

4.4. Conclusión

Con este proyecto me ha quedado claro que preparar y optimizar bien los recursos es igual de importante como hacer la programación de atrás, porque sin la interfaz no hay nada que programar para que sea funcional. También es muy importante tenerlo todo bien separado en archivos diferentes para que no haya nada hardcodeado, separarlos en colores, textos y estilos, además de que facilitan mucho la programación posterior. El uso del formato WebP ha sido un descubrimiento para mi, ver como con solo cambiar el formato reduce mucho la capacidad, y va mucho más rápido.